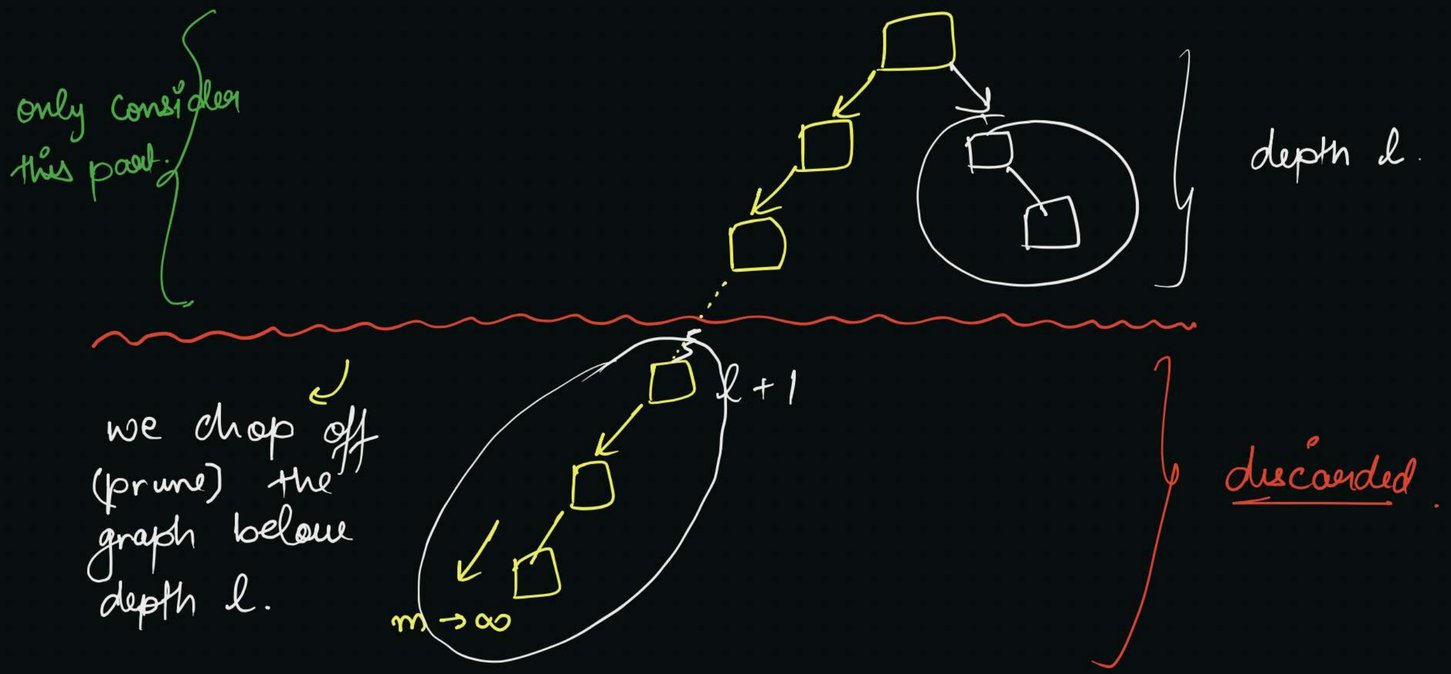
5. Uninformed Search - III

Saturday, 14 September 2024 8:40 PM

Depth - limited Search :-



↳ To avoid ∞ depth problem of DFS, we decide to search only until depth l.

↳ Treating all nodes at depth l, as if they have no successors.

Pseudo algorithm :-

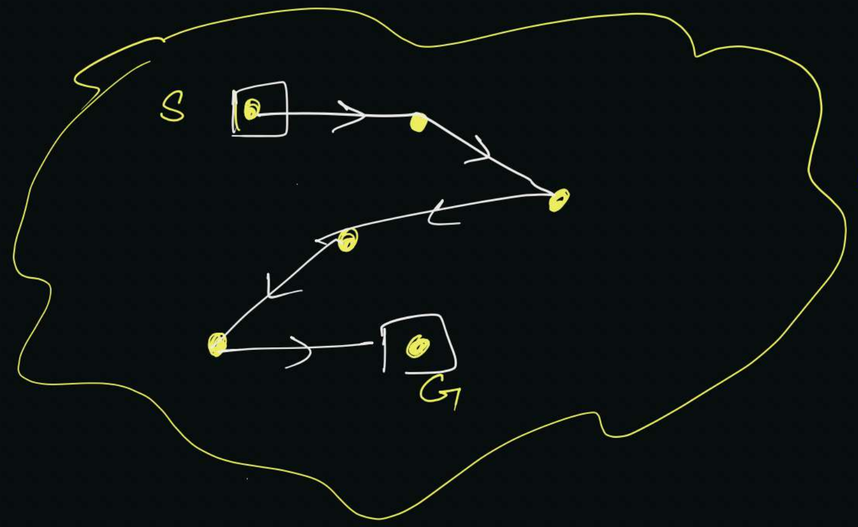
```
take s as an empty stack
s.push(start)
marks start as visited
while (s is not empty)
{
    t = s.pop()
    if (t == g) → return t ;
    else if (depth(t) > l) → continue ; → skip the successors if depth reached beyond l.
    else
    {
        for all neighbours t' of t
        {   if (t' is not visited)
            {
                s.push(t')
                marks t' as visited.
            }
        }
    }
}
```

S → Stack
start → starting node.
g → goal state.

Performance analysis :-

1. Complete :- Yes if $d < \ell$, else not

$\ell = 15$



State.

(A)

30 cities

2. Time Complexity :-

$$\text{time complexity} = O(b^{\ell})$$

; $m \equiv \ell$ = maximum depth.

3. Space :-

$$\text{Space complexity} = O(b * \ell)$$

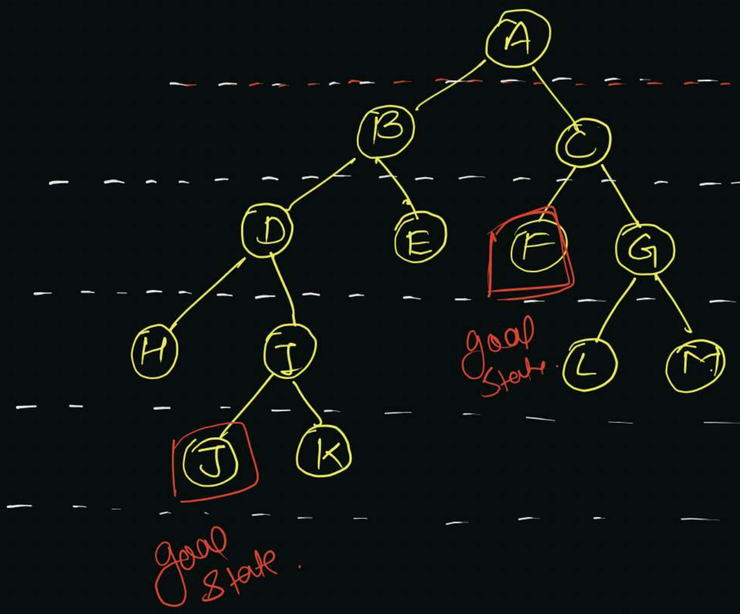
4. Optimal :- No, (Same as DFS).

Iterative - deepening search :-

↳ pseudo - algorithm :-

```
for depth, d' = 0 to ∞
{
    x = depth_limited_search (start, d')
    if (x ≡ g) → return x ;
}
```

eg.



limit = 0 I iteration

limit = 1.

limit = 2.

limit = 3

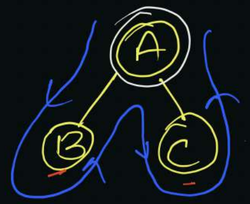
limit = 4.

Goal states = {J, F}

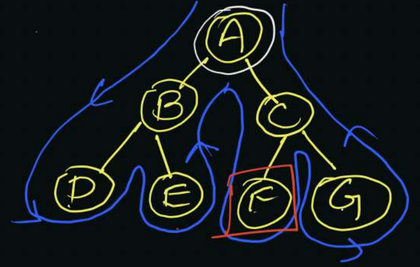
I - iteration

Start.

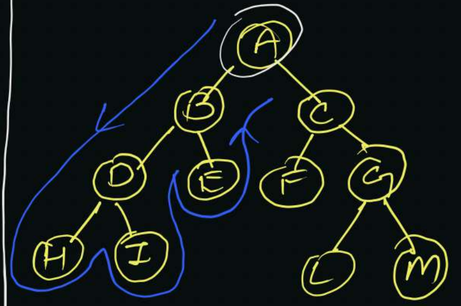
II - iteration



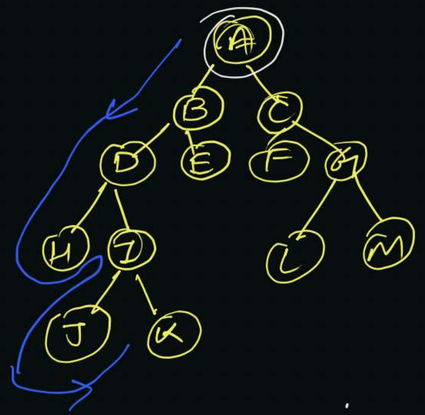
III - iteration



IV iteration



V - iteration



Performance analysis :-

1. Complete :- Yes its complete, the search will only terminate at depth d.

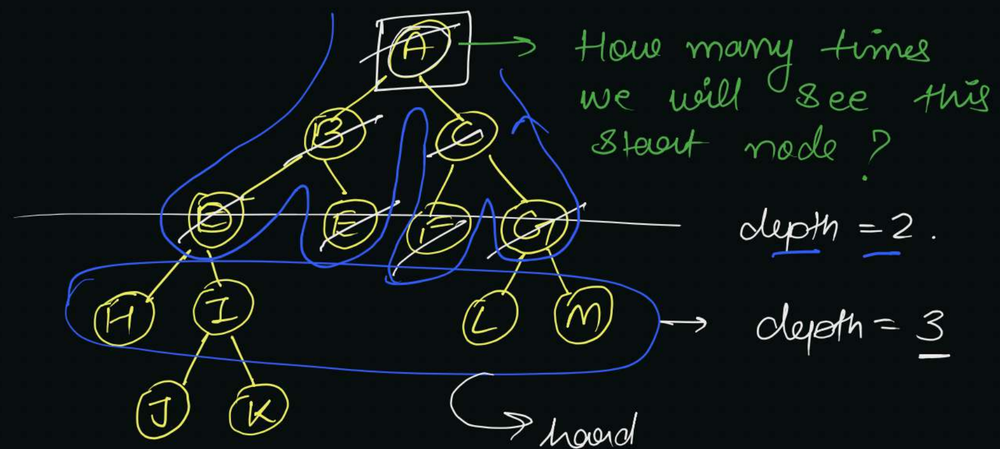
$$d \leq m$$

$d' \rightarrow 0, 1, 2, \ldots, d$

2. Time :-

$$= (d+1)\, b^0 + (d)\, b^1 + (d-1)\, b^2 + (d-2)\, b^3 + \vdots + (2)\, b^{d-1} + b^d$$

time Complexity $= O(b^d)$



In previous iteration all nodes at depth d-1 were explored.

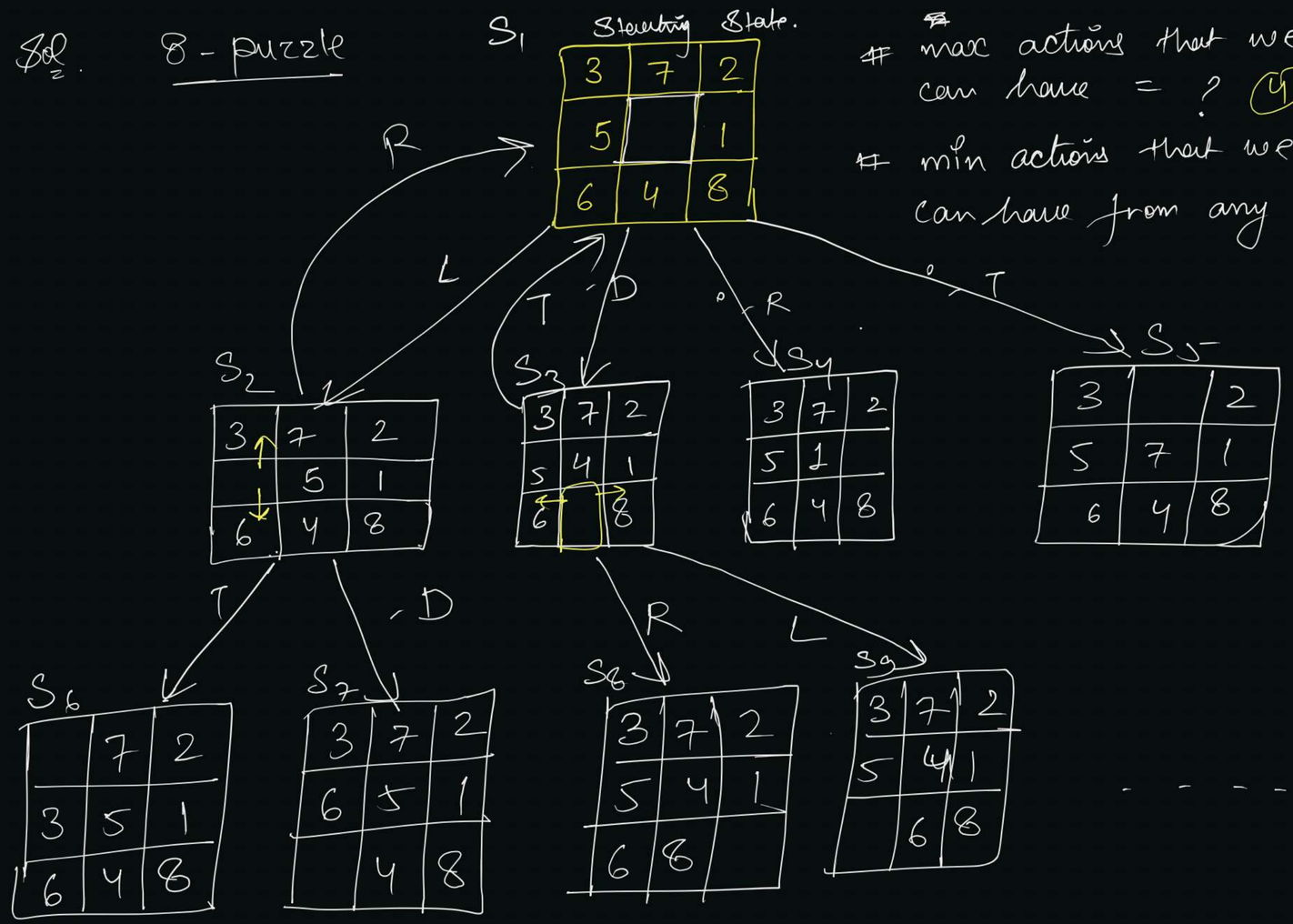
3. Space :-

Space complexity = $O(b^d)$ → applying DFS,

$m = d$

4. Optimal :- Yes, if step count = 1.

Sol. 8-puzzle



max actions that we can have = ? (4)

min actions that we can have from any state. = ? (2)

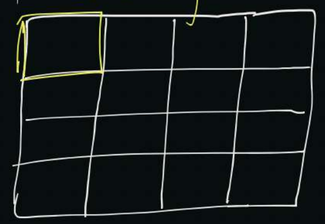
State Space = {$S_1, S_2, S_3, \ldots\ldots$}

Actions = {Shift T, Shift D, Shift R, Shift L}

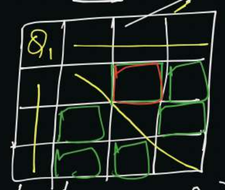
actions(S_2) = {Shift R, Shift T, Shift D}

(2.) 4 - Queens.

S_1 Starting State.

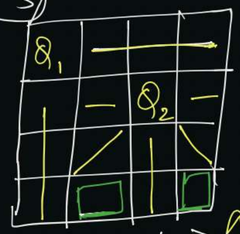


S_2



16 possible States.

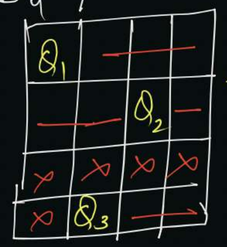
S_3



↳ 6 possible states.

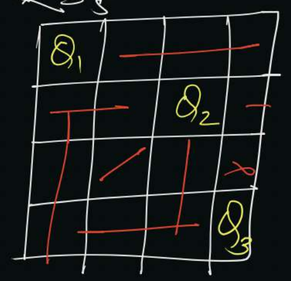
S. - - - - -

S_4



I dead end.

S_5



Goal State.